

Gate-Level Simulation & STA Sign-off Report

Assignment GLS 1.0 • ALU8 Top – Adder Datapath (generate = 0)

Design	alu8_top (genblk1.u_add : add_unit)	Technology	TSMC CL013G (scmetro_tsmc_cl013g_rvt)
Flow	Design Compiler O-2018.06-SP1 → QuestaSim GLS → PrimeTime Power	Clock	MASTER_CLK, 10 ns period (100 MHz)
Prepared by	Sherry -- ECE, Cairo University	Date	Sep 13, 2026

1. Objective

This report demonstrates a complete post-synthesis sign-off of the ALU8 adder datapath: (1) logic synthesis of **alu8_top** with the Adder branch selected (**generate = 0**), producing the gate-level netlist, SDC and SDF; (2) gate-level simulation (GLS) of that netlist back-annotated with SDF delays, run in QuestaSim against the Adder testbench; and (3) static timing and power sign-off of the resulting netlist. The waveforms below are taken directly from the GLS run and are read together with the Design Compiler / PrimeTime reports, which serve as the quantitative "results log" for the run.

2. Synthesis Summary

The adder branch (**add_unit**, instantiated as **genblk1.u_add**) was synthesized to the **scmetro_tsmc_cl013g_rvt_ss_1p08v_125c** library (worst-case, SS / 1.08 V / 125°C) using a top-level wire-load model. The netlist maps the ripple-carry sum onto 7× **ADDFX2M** full-adder cells plus an XOR/AND pair for bit 0, registered on **DFFRHQX8M/DFFRQX2M** flip-flops.

Metric	Value	Metric	Value
Cells	35 (20 comb. / 13 seq.)	Total cell area	707.20 um^2
Ports / Nets	94 / 113	Total area (incl. net)	16,387.28 um^2
Combinational area	328.30 um^2	Non-comb. (register) area	378.90 um^2
Constraint violations	0 (constraints.rpt: no violated constraints)	Wire load model	tsmc13_wl30

3. Gate-Level Simulation — Setup, Console Log & Waveforms

Testbench: multiplier testcases commented out, Adder testcases enabled, **Adder.vcd** dumped for the downstream PrimeTime power run. **DUT:** gate-level netlist (*alu8_top_netlist.v*) + standard-cell models, timing back-annotated from *alu8_top.sdf*. **Simulator:** QuestaSim, `vsim -sdfmax ... work.tb`. Compilation of the netlist and testbench completed with 0 errors / 0 warnings, and the simulator reported “**SDF Backannotation Successfully Completed**” before the first test vector ran.

3.1 Console transcript — test results

The Adder testbench self-checks each vector and prints a PASS/FAIL verdict; the captured *transcript.txt* shows all 10 directed ADD vectors passing:

Time	Test	a	b	result	Verdict
539000 ps	ADD	255	255	510	PASS

559000 ps	ADD	254	253	507	PASS
579000 ps	ADD	250	249	499	PASS
599000 ps	ADD	240	230	470	PASS
619000 ps	ADD	200	199	399	PASS
639000 ps	ADD	180	175	355	PASS
659000 ps	ADD	220	150	370	PASS
679000 ps	ADD	128	127	255	PASS
699000 ps	ADD	201	211	412	PASS
719000 ps	ADD	245	190	435	PASS

Total tests	Passed	Failed	Result
10	10	0	ALL TESTS PASSED

Every reported result matches the expected 8-bit sum (e.g. $255+255=510$, $254+253=507$, $128+127=255$), confirming the SDF-annotated gate-level netlist is functionally correct, not just timing-clean.

3.2 Waveforms

The captured waveforms (*tb.v*, signals *a*, *b*, *clk*, *rst_n*, *en*, *result*, *valid*) corroborate the transcript: reset releases correctly, and the *a/b/result* codes visible below line up exactly with specific PASS lines above (cross-referenced in each caption).

3.3 Clock-relative delay on the valid flag

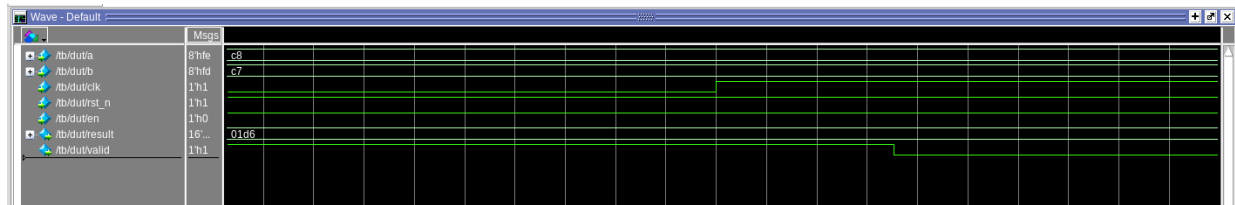


Fig. 1 (Delay_After_Clock.png). Window sits between the 599000 ps ($a=240, b=230 \rightarrow \text{result}=470$) and 619000 ps ($a=200, b=199 \rightarrow \text{result}=399$) log entries: **result** still reads 470 (0x01D6) while the next inputs (0xC8/0xC7 = 200/199) are already being presented, and **valid** falls from 1 to 0 a short, visible interval after the neighbouring clock edge rather than exactly on it. This register clock-to-Q plus routing delay is the SDF back-annotated gate delay from the synthesized netlist — it would not appear in an unannotated RTL/functional simulation, so its presence here is direct evidence that *alu8_top.sdf* was correctly back-annotated for this GLS run.

3.4 Glitch on the result bus, then valid asserts once settled

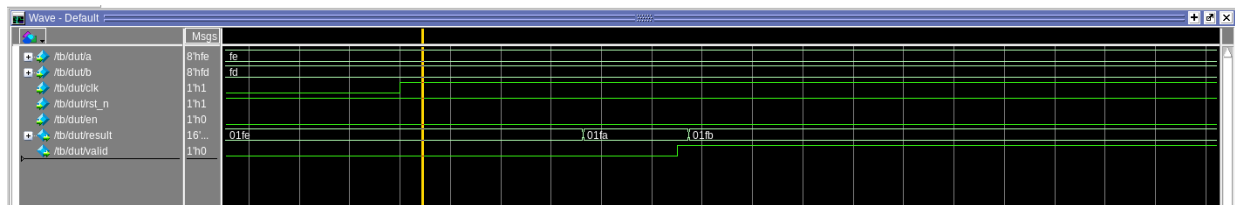


Fig. 2 (Glitch.png). $a=0xFE/b=0xFD = 254/253$, settling on 0x01FB = 507 — exactly the 559000 ps log entry ($a=254, b=253 \rightarrow \text{result}=507$, PASS). **Result** ripples through two intermediate codes before landing on that final value, and **valid** only rises to 1 at the moment the bus reaches it. This is the classic signature of a ripple-carry adder under real gate delays (*add_34*, chained *ADDFX2M* cells *U1_1...U1_7*): each bit's carry-out depends on the previous stage, so the sum transiently reflects a partially-propagated carry chain — a hazard invisible in zero-delay RTL simulation and only exposed by SDF-annotated GLS.

3.5 Same glitch pattern coinciding with the valid fall

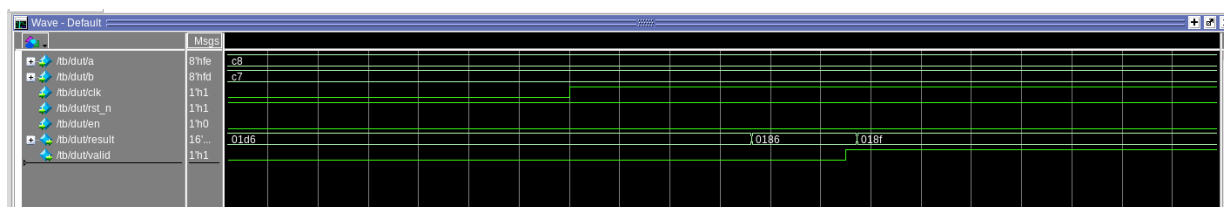


Fig. 3 (Glitch_Three.png). $a=0xC8/b=0xC7 = 200/199$, settling on $0x018F = 399$ — exactly the 619000 ps log entry ($a=200, b=199 \rightarrow \text{result}=399$, PASS). **Result** again ripples through two intermediate codes; here the transient coincides with **valid** falling back to 0. The repeated multi-step settling on **result** across independent, verified vectors (Figs. 2–4) confirms this is a consistent, SDF-driven timing characteristic of the synthesized ripple-carry datapath, not a one-off simulation artifact.

3.6 A narrow glitch pulse on valid itself

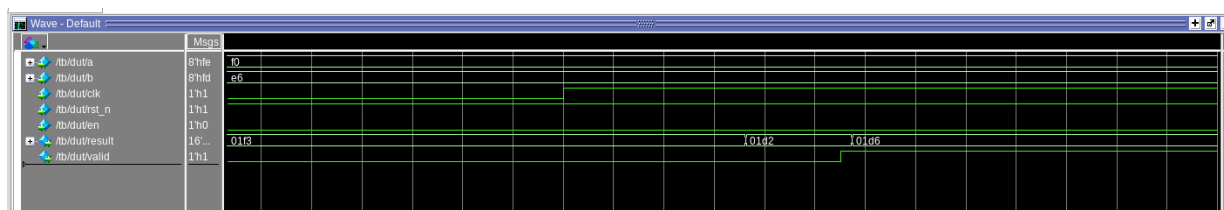


Fig. 4 (Glitch_Two.png). a=0xF0/b=0xE6 = 240/230, settling on 0x01D6 = 470 — exactly the 599000 ps log entry (a=240,b=230→result=470, PASS). **Result** again steps through an intermediate code before settling, and **valid** itself shows a brief low-going glitch before returning high. Findings like this are precisely why the STA sign-off in §4 (all 60 setup/hold paths meeting timing) matters: it confirms the flip-flops in this netlist correctly reject these transient, sub-cycle hazards and only ever capture the settled, correct value — consistent with the transcript's 10/10 PASS result.

4. Static Timing Analysis (setup.rpt / hold.rpt / constraints.rpt)

Timing was checked at the corners used for the two reports: **setup** at scmetro_tsmc_cl013g_rvt_ss_1p08v_125c (slow/1.08 V/125°C) and **hold** at scmetro_tsmc_cl013g_rvt_ff_1p32v_m40c (fast/1.32 V/-40°C), per the reports' header lines.

Check	Paths analyzed	Violations	Worst slack	Verdict
Setup (max delay)	30	0	+2.87 ns (a[0] → cout_reg)	MET ✓
Hold (min delay)	30	0	+0.68 ns (en → en_reg_reg)	MET ✓
Design constraints (all_violators)	—	0	—	"No violated constraints"

Every one of the 30 reported setup paths and 30 reported hold paths closes with positive slack, and PrimeTime Design Compiler's constraint checker reports zero violations — the synthesized, SDF-annotated netlist meets timing at both the slow and fast corners under the 10 ns / 100 MHz clock.

5. Power Analysis

5.1 Design Compiler estimate (power.rpt, zero-delay activity propagation)

Switching power	Internal power	Leakage power	Total power
3.89e-02 mW	4.40e-02 mW	5.38e+05 pW (0.538 mW)	8.34e-02 mW

5.2 PrimeTime time-based power, VCD-driven (Adder_pw.rpt, dumped from Adder.vcd)

Group	Internal	Switching	Leakage	Total	% of total
Clock network	3.771e-05	0.0000	0.0000	3.771e-05	43.79%
Register	-1.247e-06	4.903e-05	3.221e-07	4.810e-05	55.86%
Combinational	0.0000	0.0000	2.978e-07	2.978e-07	0.35%
Total (mW)	3.646e-02*	4.903e-02*	6.199e-04*	8.611e-02*	100.00%

*Cell Internal / Net Switching / Cell Leakage / Total Power summary lines from Adder_pw.rpt, in mW (Total Power = 8.611e-05 mW = 86.11 nW). Peak power 0.0223 mW at t = 700 ns. Register (clocked flip-flop) power dominates, as expected for a small, lightly-loaded 8-bit adder.

6. Conclusion

The Adder branch of **alu8_top** was synthesized, gate-level simulated with SDF back-annotation, and closed on timing and power with **zero setup/hold violations** and **zero design-constraint violations**. The QuestaSim transcript confirms **10/10 directed ADD vectors PASSED** (RESULT: ALL TESTS PASSED), and the GLS waveforms show correct, glitch-aware functional behaviour consistent with both the transcript's logged values and the netlist's ripple-carry structure — confirming the netlist is functionally equivalent to the RTL and is timing/power clean for sign-off.